Resume Ranker
The Caveman Guide

Everything about the Growth For Impact take-home project, explained from zero.
Read this once, front to back, and you will know every part of the project well enough to review it, change it and defend it in an interview.

What	Detail
Project	gfi-resume-ranker (Desktop\INTERNSHIP\gfi-resume-ranker)
For	Growth For Impact: Software Engineering Internship take-home
Deadline	14 November 2026
Built with	Next.js 14, TypeScript, Tailwind CSS, mammoth, unpdf, Vitest
Status (7 Nov 2026)	Built, reviewed, tested, runs locally. NOT deployed, NOT committed, form NOT submitted.

How to read this guide

- **Green boxes** (CAVEMAN SAYS) explain an idea in the simplest possible words.
- **Blue boxes** (SEARCH THIS) give exact words to type into YouTube or Google if a concept is new to you.
- **Yellow boxes** (WATCH OUT) point at traps, weak spots, or things an interviewer may poke at.
- Grey boxes are code or terminal commands. You do not need to memorise them.
- Sections 1-3 are the big picture. Sections 4-9 walk through the 5 build steps in order. Sections 10-15 are for reviewing, defending and finishing the project.

Contents

1. The assignment: what GFI asked for 3
2. The big picture in one page 4
3. The tools (tech stack), one by one 5
4. Folder tour: every file and why it exists 6
5. Step 1: Extraction (turning a resume into facts) 8
6. Step 2: Human review (the anti-auto-reject step) 11
7. Step 3: Scoring (how a resume gets its number) 13
8. Step 4: Classification and ranking 15
9. Step 5: The website 16
10. Tests: how we know it works 18
11. Commands cheat sheet 19
12. Domain words: LCA and UX in plain English 20
13. Decisions and why we made them 21
14. Interview prep: questions they may ask 22
15. Known limits and what to improve next 24
16. Glossary with search tags 25
17. What is done, what is left, how to review 26

1. The assignment: what GFI asked for

Growth For Impact (GFI) is building a "LinkedIn for sustainability". You applied for their Software Engineering Internship (remote, Rs 12-15k/month, web-scraping preferred). They replied with a take-home assignment, written in a Zoho Writer document called *GFI Software Engineer Assignment*. They expect at least 4 hours of focused work.

The story they gave

A company posted **two jobs** and got **47 anonymised resumes**, mixed together:

	JD1	JD2
Job	LCA Consultant (Life Cycle Assessment)	UI/UX Design Intern
Who hires	A sustainability consultancy (onsite, Chennai)	Growth For Impact itself (remote)
Experience	Brief says 4-6 yrs. JD page says 2-3 yrs (a conflict!)	0-1 year
Must-haves	Run LCA studies independently, GaBi or SimaPro, 3+ LCA projects, ISO 14040/14044	Figma, prototyping, user research, user psychology, aesthetic sense
Link	tally.so/r/9q5plp	tally.so/r/0QovpQ

What the prototype must do (6 requirements)

1. **Ingest** the 47 resumes and both job descriptions (JDs).
2. **Extract** skills, relevant experience, education, projects and role keywords. Handle missing, inconsistent or unreadable data gracefully. Keep a link to each original resume.
3. **Classify** each resume: stronger match for LCA, UI/UX, both, or neither/uncertain.
4. **Rank** resumes separately for each role, with a clearly defined score and an understandable explanation.
5. **Review UI**: both job profiles with ranked results, a candidate detail view, sorting and filtering (score, experience), and clear display of extraction or classification errors.
6. **Upload**: a reviewer uploads a new resume and sees where it ranks in the pile.

Users are **non-technical HR people**, so the screens must be simple. It must be hosted on a **free Vercel** instance.

> **WATCH OUT:** Two auto-reject rules: (1) if the AI-extracted data is not actually cleaned up and verified by you, and (2) if the methodology is not shared. That is why this project has a whole human review step (section 6) and a detailed README plus method text (section 14).

What you submit at the end

- A Google-style form: email, phone, website (github.com/SoAkeD1), earliest start date, the Vercel link, and the methodology / ranking logic text.
- A short quiz (the "submit here" link in the doc).
- Both are done by you, after deployment.

2. The big picture in one page

> **CAVEMAN SAYS:** Big pile of resumes go in. Computer read each one, pull out facts, write facts in a list. Human check the list and fix mistakes. Computer give each resume two scores (one per job) using fixed rules. Website show the ranked lists. New resume uploaded? Same rules, see where it lands.

```
47 Word resumes            JD1 + JD2 written as settings (config/jobs.ts)
      |                                  |
      v                                  |
(1) EXTRACT        npm run extract        |
    read text -> find sections -> jobs + dates, skills,
    degrees, projects, problems  ==>  data/extracted.json
      |
      v
(2) HUMAN REVIEW   npm run review:export  ==>  data/review.csv   (you check / fix)
                   npm run review:apply   ==>  data/verified.json + data/review-log.json
      |
      v
(3) SCORE + (4) CLASSIFY + RANK    npm run score   <------------------+
                                   ==>  data/results.json
      |
      v
(5) WEBSITE (Next.js)  reads results.json, shows rankings, candidate pages, problems
      +  /api/rank : upload a resume -> same steps 1, 3, 4 -> "would rank #N of 48"
```

One command runs steps 1 to 4: `npm run pipeline`. The website is then built from the finished data. Nothing is calculated when HR opens a page, except for uploads.

Why rules and not AI or machine learning?

- **No training data.** Machine learning learns from examples with answers ("this candidate was hired"). We have 47 resumes and zero answers. Nothing to learn from.
- **Must be explainable.** GFI wants a clear score and a reason. A rule like "SimaPro used in a job = +8 points" can be explained. A neural network's number cannot.
- **Repeatable.** Same resume in, same score out, every time. An LLM (like ChatGPT) may answer differently each run.
- **AI was still used**, as a coding helper (Claude) and to suggest fixes during the review. Every suggestion was checked by you against the original resume. That is what the brief means by "AI is expected, but verify it".

> **SEARCH THIS:** "rule based vs machine learning" and "what is explainable AI" (YouTube, any 5-10 min video).

3. The tools (tech stack), one by one

You already know some of these from your Ascend project. Here is what each tool does *in this project*.

Tool	What it is	What it does here	Search this
Node.js	Runs JavaScript outside the browser	Runs the scripts and the web server	Node.js in 100 seconds (Fireship)
npm	Node's package manager	Installs libraries; runs commands like npm run score	npm explained for beginners
TypeScript	JavaScript with types	All code. Types catch mistakes, e.g. a score that is text instead of a number	TypeScript in 100 seconds
Next.js 14 (App Router)	React framework for websites	The website: pages, the upload API, building static pages	Next.js App Router tutorial; nextjs.org/learn
React	Library for building UI from components	Every page is built from components like ScoreBar, LabelBadge	react.dev/learn
Tailwind CSS	Styling with class names	All colours, spacing, layout (className="rounded-xl p-4")	Tailwind CSS in 100 seconds
mammoth	Library that reads .docx files	Turns each Word resume into plain text	npm mammoth docx
unpdf	pdf.js packaged for servers	Reads uploaded PDF resumes	unpdf github; how pdf.js extracts text
tsx	Runs TypeScript files directly	Runs scripts: npx tsx scripts/extract.ts	tsx typescript execute
Vitest	Testing tool	38 tests check dates, extraction, scoring	Vitest tutorial; unit testing explained
Vercel	Hosting company (made Next.js)	Will host the site for free	deploy Next.js to Vercel
Git / GitHub	Version control / code hosting	Save history; reviewers read the code	Git and GitHub for beginners

> **CAVEMAN SAYS:** Node = engine. npm = shop for parts. TypeScript = careful JavaScript. Next.js = house frame for website. React = Lego bricks for screens. Tailwind = paint. mammoth/unpdf = eyes that read Word/PDF. Vitest = checker. Vercel = land where house is built. Git = time machine for code.

4. Folder tour: every file and why it exists

Project root: `C:\Users\shrim\OneDrive\Desktop\INTERNSHIP\gfi-resume-ranker`

The brain (lib/ and config/)

File	Job in one line
config/jobs.ts	JD1 and JD2 as settings: must-have groups, bonus skills, target years, preferred degree fields, and the fixed "today" date (Nov 2026).
lib/skills.ts	The skill dictionary: 37 skills, each with spellings/synonyms (patterns). Also the "signal words" that make a job count as LCA work, design work, or sustainability work.
lib/dates.ts	Reads date ranges like "Sep '25 - Dec '25" and turns them into months. Merges overlapping jobs.
lib/extract.ts	The extractor. Text in, structured resume out (jobs, skills, education, projects, flags). Also reads .docx / .pdf files into text.
lib/review.ts	Applies your corrections from the review sheet and logs every change.
lib/csv.ts	Tiny CSV reader/writer for the review sheet (handles commas and quotes inside cells).
lib/score.ts	The scoring rules. Gives each part points + reasons + evidence quotes.
lib/classify.ts	Turns two scores into a label (Fits LCA, Fits UI/UX, Both, Neither, Needs a look) and ranks.
lib/types.ts, lib/data.ts	Shapes of the data, and the loader the website uses to read results.json.

The pipeline (scripts/)

Script	npm command	What it does
scripts/extract.ts	npm run extract	Reads all 47 resumes, writes data/extracted.json, prints a summary table.
scripts/export-review.ts	npm run review:export	Writes data/review.csv sorted by risk. Keeps your earlier edits.
scripts/apply-review.ts	npm run review:apply	Merges your edits into data/verified.json and data/review-log.json.
scripts/score.ts	npm run score	Scores, labels and ranks everyone into data/results.json. Prints top 10s.
scripts/set-review.ts	npx tsx scripts/set-review.ts ID col=value	Helper used during review to write a decision into review.csv without Excel.
scripts/dump-text.ts	npx tsx scripts/dump-text.ts	Dev helper: dumps raw text of every resume into data/text/ (not committed).

The data (data/ and public/resumes/)

File	What is inside
public/resumes/*.docx	The 47 original resumes. In public/ so the website can offer "Open original resume".
data/extracted.json	Raw extractor output for all 47 (before human review).
data/review.csv	The review sheet (open in Excel). Your decisions live in the corrected_* / review_status / reviewer_notes columns.
data/verified.json	extracted.json + your corrections. Scoring uses only this.
data/review-log.json	Proof of review: every checked resume, its status, notes, and every change (old -> new).
data/results.json	Final: verified data + scores + reasons + labels + ranks. The website reads this.

The website (app/ and components/)

File	Page / role
app/layout.tsx	The frame around every page: top bar with Rankings, Upload, Problems, How scoring works.
app/page.tsx	Home "/": the two job tabs and the ranked table.
app/candidate/[id]/page.tsx	Candidate detail "/candidate/HM-4257".
app/issues/page.tsx	"/issues": every data problem and every "Needs a look" resume.
app/upload/page.tsx	"/upload": drag-and-drop a resume, see its result.
app/api/rank/route.ts	The server function the upload page calls (POST /api/rank).
app/how-it-works/page.tsx	Plain-English scoring rules for HR.
components/RankingTable.tsx	The table with search, sort, sliders, label filters (runs in the browser).
components/CandidateView.tsx	Everything on the candidate page; reused for upload results.
components/ScoreCard.tsx	One role's score breakdown with "Show proof from the resume".
components/ui.tsx	Small parts: LabelBadge, ScoreBar, Card, FlagIcon, ReviewedTick.

Other files

- **tests/**: dates.test.ts, extract.test.ts, score.test.ts (38 tests, section 10).
- **README.md**: the methodology for GFI reviewers. **docs/methodology.md**: the text to paste into the form.
- **package.json**: library list + npm commands. **next.config.mjs**: tells Next.js to keep mammoth and unpdf as normal Node libraries. **tsconfig.json**: TypeScript settings. **tailwind.config.ts**: Tailwind settings.
- **node_modules/** and **.next/**: downloaded libraries and build output. Never edit, never commit.

5. Step 1: Extraction (turning a resume into facts)

> **CAVEMAN SAYS:** Me open resume box. Me pull words out. Me cut words into piles: jobs, school, skills, projects. Me find dates, count months. Me find skills from my list. Me write it all down. If something missing, me put red flag, me NOT guess.

5.1 What is a .docx file?

A .docx is secretly a **zip folder** full of XML files. The actual text lives in `word/document.xml`. The library **mammoth** opens the zip and gives us plain text, one paragraph per line. You can try it: rename a copy of any .docx to .zip and open it.

> **SEARCH THIS:** "what is inside a docx file" / "Office Open XML explained".

5.2 What the resumes look like

All 47 resumes were generated from one template, so they share headings. A typical one (TM-4495):

```
TM                                          <- initials (the anonymised name)
Sustainability Engineer                     <- headline
Experience                                  <- section heading
Engineer – L&T Technology Services   March 2023 – Present     <- job line: Title – Company + 3 spaces + dates
• Performed Life Cycle Assessment using SimaPro for EV and ICE platforms...
Projects / Deliverables
Interior System LCA – Instrument Panel      <- project title
Performed Life Cycle Assessment of vehicle instrument panel assemblies using SimaPro...
Education
M.Tech in Automotive Engineering
MS Ramaiah University of Applied Science
Professional Summary
Sustainability engineer with 3 years of experience in Life Cycle Assessment (LCA)...
Core Skills
Platforms: SimaPro, Siemens NX, MATLAB/Simulink, MS Excel...
```

Headings found across the 47: Education (46), Core Skills (46), Experience (44), Professional Summary (43), Projects / Deliverables (32), Tools / Technologies (30), Certifications and Licenses (22), Languages (15), Awards / Honors (13).

5.3 Splitting into sections

The extractor walks line by line. If a line matches a known heading (several spellings accepted, e.g. "Work Experience", "Employment History"), everything after it belongs to that section until the next heading. Lines before the first heading are the **header** (initials + headline). If fewer than 2 sections are found, the resume gets a warning flag.

5.4 Regex: the pattern finder

Almost all extraction uses **regular expressions** (regex): tiny patterns that describe text. Examples from this project:

Pattern	Means	Matches
/sima\s?pro/i	"sima", optional space, "pro"; i = ignore case	SimaPro, Sima Pro, simapro
/\bLCAs?\b/	whole word LCA or LCAs (case-sensitive)	LCA, LCAs, but not "LCAnalysis"
/cradle[\s-]to[\s-](gate|grave)/i	cradle, space or dash, to, gate or grave	cradle-to-gate, Cradle to grave
/(\d+)\s*\+?\s*years? ... experience/i	a number, optional +, year(s) ... experience	"3+ years of experience"
/(\d+)\s*EPDs\b/i	a number then EPDs	"Published 40 EPDs"

Key symbols: `\b` word edge, `\s` space, `\d` digit, `?` optional, `+` one or more, `*` zero or more, `(a|b)` a or b, `[ab]` one of these characters, `i` ignore case.

> **SEARCH THIS:** "Regular expressions in 100 seconds" (Fireship) and practise on regex101.com. This is the single most important topic in the project.

> **WATCH OUT:** Acronyms are matched case-sensitively on purpose. "UI" in capitals is a skill; "ui" inside "build" is not. Even so, a candidate whose initials were "UI" got a UI-design skill. That bug was caught in review (section 6.5).

5.5 Dates and years of experience (lib/dates.ts)

Each job line ends with a date range. The resumes use about 15 different styles:

Written as	Read as	Notes
Jan 2019 – Dec 2019	12 months	Both ends counted (inclusive)
March 2023 – Present	Mar 2023 to Nov 2026 = 45 months	"Present" = fixed date Nov 2026, so results are repeatable
Sep '25 – Dec '25	Sep 2025 to Dec 2025	Two-digit year with apostrophe
Jan 26 - April 26	Jan to Apr 2026, flagged	"26" could be a day; assumed 2026 and flagged
09/2023 – 09/2024	13 months	Month/year
May – July 2025	May to Jul 2025	Start month borrows the end year
2016 - 2019	36 months, flagged	Year only: month assumed
8 Months	8 months, flagged	Only a duration, no dates
Jun 2025 - Dec 2026	capped at Nov 2026, flagged	End date in the future
null / Not specified / Ongoing	0 months, flagged	Never guessed

Overlapping jobs are counted once

If someone had two jobs at the same time, adding them would double-count. So every job becomes a block of months on a timeline, overlapping blocks are merged, and then the months are counted.

```
Job A: Jan 2020 ------------ Dec 2020           (12 months)
Job B:        Jun 2020 ------------ Jun 2021    (13 months)
Merged: Jan 2020 ------------------ Jun 2021    = 18 months  (not 25)
```

> **SEARCH THIS:** "merge overlapping intervals" (a classic coding interview problem, LeetCode 56).

Three kinds of years

- **Total years**: all jobs merged.
- **Relevant years per role**: only jobs whose title or bullets contain that role's signal words. LCA signals: life cycle, LCA, LCI, SimaPro, GaBi, EPD, carbon footprint, cradle-to. Design signals: UI, UX, user experience, product designer, Figma, wireframe, prototype, interaction design, user research.
- **Sustainability years**: jobs about ESG, GHG, carbon, emissions, CBAM, climate, environment, circular economy, decarbonisation. Used for LCA scoring at half weight.

The resume's own claim ("3+ years of experience") is also read. If it clearly disagrees with the dates, a warning is shown. "3+" means *at least 3*, so 4.5 calculated years is **not** a conflict.

5.6 Skills: used vs only listed

lib/skills.ts holds 37 skills (19 LCA, 18 UI/UX). Each skill has several patterns. For every skill found, the extractor records:

- **usedInWork**: found inside a job or project. Strong evidence. Counts 100%.
- **listedAsSkill**: found only in Skills, Tools, Summary or Certifications. Weaker. Counts 60%.
- **evidence**: up to 3 quotes (the exact line), shown on the website under "Show proof from the resume".

> **CAVEMAN SAYS:** Saying "me know SimaPro" in skills list = some points. Saying "me did LCA of car parts using SimaPro" in a job = full points. Doing beats saying.

5.7 Counting LCA projects

The JD wants at least 3 completed LCA projects. The count is the highest of: (a) projects whose title or description mention LCA/EPD/LCI, plus jobs with an LCA bullet (one each), and (b) a stated number like "Published 40 EPDs" or "5 LCA projects". An EPD (Environmental Product Declaration) is itself the output of an LCA study, so each EPD counts as one.

5.8 Education

Degree lines are found with a pattern of degree words (B.Tech, M.Tech, B.Des, MBA, Master, Bachelor, PhD, Diploma...). Each degree gets a level (phd, master, bachelor, diploma, school), a field (text after "in", a dash or "of"), the institution (next line) and the end year. "Expected" or a future year marks it in progress.

5.9 Flags: never guess, always say

Level	Example messages
error	File is empty or unreadable: no text could be extracted.
warning	Dates missing; only a duration given; end date in future; year-only dates; 2-digit year; no job has readable dates; resume says X years but dates add up to Y; very short resume; no clear sections; no degree found; no skills found; looks like a duplicate.
info	No Experience section (may be a fresher); some numbers look removed by anonymisation (ISO 14040 became "ISO /").

5.10 Real traps in this data set

- **One truly empty file**: anonymized_resume_1793685080327.docx. Its page, comments and footnotes are blank and there are no images. It stays in the list as "Needs a look" with score 0, so HR can ask for a resend.
- **Anonymisation ate numbers**: "ISO 14040/14044" became "ISO /, ISO", "Scope 1, 2 and 3" became "Scope s", even "outstanding" became "anding". So ISO cannot be checked for anyone; it is not scored (decision in section 13).
- **Portfolio links removed**, so the portfolio bonus is almost never earned.
- **Typos in dates**, e.g. an internship ending "Dec 2026" (future).

5.11 What came out

47 resumes read. 37 clean, 9 with warnings, 1 error (the empty file). About 20 sustainability/LCA people and about 27 designers.

6. Step 2: Human review (the anti-auto-reject step)

> **CAVEMAN SAYS:** Machine read fast but make mistakes. Human check machine homework, fix wrong answers, write down every fix. Written list of fixes = proof for GFI that human checked.

6.1 How it works

1. `npm run review:export` writes **data/review.csv**: one row per resume with everything extracted, plus empty columns for corrections.
2. Rows are sorted by risk: **priority 1** = has errors/warnings; **priority 2** = suspicious pattern (LCA mentioned but 0 LCA years, stated years differ, designer with no projects); **priority 3** = the rest. A check_reason column says why.
3. You compare a row with the original resume and either type **ok** in review_status or fill a correction column: corrected_totalYears, corrected_lcaYears, corrected_uiuxYears, corrected_lcaProjects, add_skills, remove_skills, corrected_eduField. Plus a note in reviewer_notes.
4. `npm run review:apply` merges the edits into **verified.json** and writes **review-log.json** (old value -> new value, per field). Scoring only ever reads verified.json.
5. Re-running export keeps your edits, so nothing is lost if extraction is improved later.

6.2 What we actually did (7 Nov 2026)

We went through the resumes in batches. For each one, Claude showed the extracted values next to the original text and suggested a decision; **you chose**. Then Claude wrote your decision into review.csv with a note.

Group	Resumes	Result
Priority 1 (warnings/errors)	AK-4184, AR-0207, AY-0440, BA-4206, FK-0451, FT-0301, KA-4307, KN-0310, MI-0235, resume-0327, SS-4484, VM-4520	All checked
Priority 2 (suspicious)	AA-0195, AA-4144, BS-0461, DP-0247, NA-4346, PKG-4369, RS-4414, UI-4506	All checked
Top 5 of each ranking	HM-4257, TM-4495, SRK-4455, SS-4470, MS-4442, CG-0278, DVJ-0268, APS-0222, MM-0338, AA-0402	All checked
Not checked	17 others (no warnings, not top 5)	Shown as "not checked" in the app

Totals: 30 of 47 checked. 27 matched. 3 corrected.

6.3 The 3 corrections

Resume	Change	Why
AK-4184	removed skills portfolio, responsive	They came from "Developed a responsive personal portfolio website (HTML/CSS)" in IT training: web coding, not UX design.
BS-0461	design years 0.8 -> 1.0	"Product Design Intern - Techlanz" designed 20+ web interfaces but had no UI/UX keywords. Founder role (3D motion) left out.
SRK-4455	LCA projects 2 -> 3	Leads LCAs for "major UK and global clients" since 2022 plus LCAs at IIT Madras. No number written, but clearly 3+.

6.4 Where we refused to guess

- **AR-0207**: only job has dates written "null". Kept 0 years.
- **SS-4484**: PwC job says only "Ongoing". Probably started 2026, but that is a guess, so not used. LCA score 49.2 -> "Needs a look".
- **KN-0310**: 640-character resume, relevant UI/UX postgraduate, no projects. Kept as "Needs a look" for HR to read.
- **resume-0327**: truly empty. Kept in the list, not deleted.

6.5 Bugs the review found (fixed in code for everyone)

Important idea: when review finds a *pattern* mistake, fix the **rule**, not just that one row. Otherwise the same mistake stays hidden in other resumes and in every future upload.

#	Bug	Fix
1	Candidate with initials "UI" got a UI-design skill	Skip the initials line when matching skills
2	"MS Ramaiah University" read as an MS degree	Lines that look like institutions are not degrees
3	"Bachelor of Technology in CS" gave field "Technology"	Prefer the text after "in" over "of"
4	"Project Associate (UNDP - supported)" cut at the dash	Ignore dashes inside brackets
5	"3+ years" flagged when dates showed 4.5	"+" means at least
6	LCA software Air.e not recognised	New skill: other LCA software (Air.e, Umberto, One Click LCA)
7	Circular-economy research not counted as sustainability	Added circular economy and decarbonisation to signal words
8	"Electronics Communication Engineering" got the design-field bonus	Field rule now needs mass/visual/design communication
9	Designers' projects described by outcome ("truck aggregator app", "90% task success") were not counted as design work (8 designers affected)	For a clear designer (design headline or degree), app / users / dashboard / interface / usability words also count

Before the review, three more false matches were fixed: engineering "product design" counted as UI/UX; car "usability" counted as UX testing; a finance "multi-product portfolio" counted as a design portfolio. Also: "Published 40 EPDs" was not counted as LCA projects (now it is), and goal-and-scope / functional-unit language was added as an LCA method skill.

> **SEARCH THIS:** "human in the loop machine learning" and "data validation best practices".

7. Step 3: Scoring (how a resume gets its number)

> **CAVEMAN SAYS:** Each resume get two report cards, one per job. Each card has 4 subjects. Add the marks, get a score out of 100. Every mark has a written reason and a quote from the resume.

7.1 LCA Consultant: 100 points

Part	Max	Rule
Skills match	40	Must-haves are 60% (24 pts): 3 groups x 8 pts: Life Cycle Assessment; GaBi or SimaPro; LCI data work. Used in work = 8, only listed = 4.8. Bonus skills are 40% (16 pts): EPD, LCA method, openLCA, other LCA software, ecoinvent, carbon footprint, cradle-to-gate, LCIA, benchmarking, GHG, circular economy. Each used = 2.67, listed = 1.6, max 16.
Relevant experience	25	effective years = LCA years + 0.5 x (sustainability years - LCA years). 4 to 6 yrs = 25. Below 4 = 25 x years / 4. Above 6 = 23.
LCA projects	20	20 x min(count, 3) / 3. So 1 = 6.7, 2 = 13.3, 3+ = 20.
Education	15	PhD or Master's 10, Bachelor's 8, Diploma 5, school 0; +5 if any degree is in an environmental, sustainability, energy, chemical, civil, agricultural, mechanical or materials field. Max 15.

7.2 UI/UX Design Intern: 100 points

Split: **skills 40 · design work 40 · experience fit 15 · education 5**. The JD is about tools and output (its 90-day goals are a website revamp and a job board) and never mentions a degree. The first version copied the LCA split (design work 30, education 15); it was changed because it did not match the JD.

Part	Max	Rule
Skills match	40	Must-haves 24 pts: 4 groups x 6: Figma; Prototyping; User research or user psychology or usability; UI/UX design principles. Used = 6, listed = 3.6. Bonus 16 pts: wireframing, user flows/IA, design systems, accessibility, typography/visual craft, portfolio, responsive, Framer/ProtoPie.
Design projects and portfolio	40	8 per design project or design job, max 4 pieces = 32; +8 if a portfolio is mentioned.
Experience fit	15	Internship wants 0-1 yr. Up to 1.5 yrs design work = 15. 0 yrs but design background = 9. 1.5-3 yrs = 12. Over 3 = 8. No design background = 0.
Education	5	The UI/UX JD never asks for a degree, so only 5: any degree 2 (Bachelor's in progress included), Diploma 1; +3 for design, HCI, interaction, visual, fine arts, animation, architecture, media, mass/visual communication, journalism, psychology.

Special case: an unreadable file scores 0 for both roles (it does not get default points).

7.3 Worked example: TM-4495 for LCA = 87.1

Part	Points	Why
Skills	28.3 / 40	LCA used in work (+8). SimaPro used in work (+8). LCI data work missing (0). Bonus: carbon footprint, cradle-to-gate, LCIA, benchmarking, circular economy (+12.3).
Experience	23.8 / 25	3.8 yrs of LCA work (Mar 2023 to Nov 2026). Just under the 4-yr target: 25 x 3.8 / 4 = 23.75.
LCA projects	20 / 20	3 LCA projects + 1 LCA job = 4, which is 3 or more.
Education	15 / 15	M.Tech (+10), mechanical engineering field (+5).
TOTAL	87.1	Rank #2 for LCA. Label: Fits LCA.

7.4 Worked example: CG-0278 for UI/UX = 89

Part	Points	Why
Skills	40 / 40	All 4 must-haves used in work (4 x 6 = 24). 6 bonus skills, capped at 16.
Design work	32 / 40	3 design projects + 1 internship = 4 pieces x 8. No portfolio link (removed by anonymisation).
Experience fit	15 / 15	0.3 yrs design work: ideal for an intern.
Education	2 / 5	B.Tech (+2). Electronics and Communication is not a design field, so no +3 (this was bug #8).
TOTAL	89	Rank #2 for UI/UX. DVJ-0268 (B.E. Computer Science and Design) is #1 with 90.9.

7.5 The ideal-candidate check (and the flaw it found)

A good way to test any scoring system: write the **perfect** candidate straight from the JD and check that it scores exactly 100, and write a clearly **wrong** candidate and check that it scores low. These are now automatic tests (tests/perfect.test.ts).

The check found a real design flaw: a perfect UI/UX **intern** with a Bachelor's could only reach **98**, because education gave Master's 10 but Bachelor's 8 for both jobs. Interns rarely have a Master's, so that was unfair. Education points are now set per job in config/jobs.ts: for the internship a Bachelor's (finished or in progress) gets full level points; the LCA consultant role keeps Master's 10 / Bachelor's 8. Later the whole UI/UX education part was cut to 5 points (the JD asks for no degree) and design work raised to 40. Follow-up decision: with identical work, a Master's should still rank above a Bachelor's, so the degree became the first **tie-breaker** (section 8) instead of extra points.

> **CAVEMAN SAYS:** Before building a ruler, describe the perfect thing it should measure as 100. Then check the ruler says 100. We did it late, it caught a mistake, now it is a test.

> **WATCH OUT:** Every number above (8 points, 60%, half weight, 4-6 years, 50 line) is a **judgement call**, not a law of nature. Be ready to explain why you chose it (section 13) and to say it is easy to change in config/jobs.ts and lib/score.ts.

8. Step 4: Classification and ranking

Website label	Internal	Rule
Fits LCA	LCA	LCA score 50 or more, UI/UX below 50
Fits UI/UX	UI/UX	UI/UX 50 or more, LCA below 50
Fits both	Both	Both 50 or more (nobody in this pile)
Fits neither	Neither	Both below 50
Needs a look	Uncertain	File unreadable; OR a score within 5 points of 50 and the label would be Neither or Both; OR 3+ warnings near the line; OR very short resume with some signal (best score 30+) that would be Neither

Ranking: sort by score, high to low. On an **equal score, the higher degree ranks first** (PhD > Master's > Bachelor's > Diploma), then more relevant years, then ID, so the order never changes randomly. A degree never beats a higher score: 79.9 with a PhD still ranks below 80 with a Bachelor's. Uploads use the same rule.

> **CAVEMAN SAYS:** Two people, same everything, same score: the one with the bigger degree goes first. But better work always beats a bigger degree.

Results today

Label	Count	Who
Fits LCA	11	HM-4257, TM-4495, SRK-4455, SS-4470, MS-4442, VM-4520, NP-4325, PW-4228, RR-4428, HD-4275, UI-4506
Fits UI/UX	23	Most designers
Fits neither	8	Environmental / ESG / compliance people without real LCA work, e.g. BA-4206, KA-4307, RS-4414
Needs a look	5	AK-4184 (LCA 45.1), SS-4484 (49.2), VG-4531 (46.5), KN-0310 (very short), resume-0327 (empty)

Rank	LCA Consultant	UI/UX Design Intern
#1	HM-4257: 88.4 (LCA Analyst, 4 yrs, 40 EPDs)	DVJ-0268: 90.9
#2	TM-4495: 87.1 (3 SimaPro LCAs)	CG-0278: 89
#3	SRK-4455: 84.5 (LCA consultant since 2022)	APS-0222: 82.6 (M.Des)
#4	SS-4470: 75.3	AA-0402: 80.5
#5	MS-4442: 73.8	MM-0338: 79.9

9. Step 5: The website

> **CAVEMAN SAYS:** Website is shop window. Shop window only show finished goods (results.json). Only the upload counter does live work.

9.1 Next.js ideas you need

Idea	Plain meaning	In this project
App Router	The folder path is the web address	app/issues/page.tsx = /issues; app/candidate/[id]/page.tsx = /candidate/HM-4257
[id] folder	A dynamic route: one file serves many addresses	47 candidate pages from one file
Server component	Runs on the server, sends ready HTML; can read files	Home, candidate, issues, how-it-works pages read results.json
Client component	Runs in the browser; can react to clicks	RankingTable (filters), upload page. Marked with "use client" at the top
Static generation	Pages built once at build time	generateStaticParams builds all 47 candidate pages ahead of time: fast and free
searchParams	The ?key=value part of the address	/?role=uiux opens the UI/UX tab; /candidate/X?role=lca
Route handler (API)	A server function at an address	app/api/rank/route.ts answers POST /api/rank
layout.tsx	Wraps every page	Top navigation and footer

> **SEARCH THIS:** "Next.js App Router crash course", "React server components vs client components", "Next.js generateStaticParams". Official free course: nextjs.org/learn.

9.2 The pages

Page	What HR sees and can do
Rankings /	Two tabs (LCA Consultant, UI/UX Intern) showing each job summary. A ranked table: rank, candidate, score bar, best-fit badge, relevant years, top 3 skills, missing must-haves, problems count, "checked" tick. Search box, sort (best match / most experience), minimum score slider, minimum years slider, label filter buttons, "only show resumes with problems", reset. Click a row to open the candidate. On phones the table becomes cards.
Candidate /candidate/[id]	Header with label and the reason for it, 4 stat boxes, "Open original resume" download. Data problems. Human review status, notes and changes. Two score cards (chosen role first) with bars per part, reasons, and "Show proof from the resume". Extracted jobs (tagged LCA work / Design work / Sustainability), education, projects, skills (dark = used, light = listed), certifications.
Problems /issues	All "Needs a look" resumes with reasons, then every flag in a table with the reviewer's note.
Upload /upload	Drag and drop or choose a .docx/.pdf (max 4 MB). Shows "#N of 48" for each job, the label, and the full candidate view. Nothing is saved.
How scoring works	All rules in plain English, the JD years conflict, known limits.

9.3 What happens when you upload

```
Browser (upload page)                     Server (app/api/rank/route.ts)
---------------------                     ------------------------------
pick file  --- POST /api/rank (FormData) -->  check: is it .docx/.pdf? under 4 MB?
                                              .docx -> mammoth -> text
                                              .pdf  -> unpdf   -> text (lines rebuilt from positions)
                                              extractFromText()  (same code as step 1)
                                              scoreBoth()        (same code as step 3)
                                              classify()         (same code as step 4)
                                              rank = 1 + (pile resumes with a HIGHER score)
                                              same text as a pile resume? add a note
show result <------------- JSON -----------   (nothing stored)
```

Example: a pile resume re-uploaded as "new-candidate.docx" got LCA 75.3, **#4 of 48**, with the note that it matches SS-4470.

> **WATCH OUT:** Uploaded resumes are **not** human-reviewed (there is no time to). Their review box is hidden. Say this if asked.

9.4 The PDF problem we hit

The first PDF library (pdf-parse) failed on every modern PDF with "Illegal character: 41" because it bundles a 2018 version of pdf.js. It was replaced by **unpdf** (current pdf.js for servers). pdf.js gives text pieces with x/y positions but no line breaks, so the code starts a new line whenever the y position changes. A test PDF then scored UI/UX 75.2 (#7 of 48).

10. Tests: how we know it works

A **unit test** is a tiny program that runs one function with a known input and checks the output. Run them all with `npm test`. Currently **45 tests pass**.

File	Checks (examples)
tests/dates.test.ts (19)	12 real date formats give the right months; future end dates capped; future start not counted; null/Not specified flagged; "8 Months" read; 2-digit years flagged; overlapping jobs merged; gaps not filled.
tests/extract.test.ts (12)	Short ID, header, both jobs and companies, total and LCA years, used vs listed skills, LCA project count, education; empty file flagged; regression tests for bugs 1, 2, 4 and 9.
tests/score.test.ts (11)	Parts add up to the total and stay within max; LCA expert high for LCA and low for UI/UX; design intern the reverse; reasons exist; correct labels; unreadable = 0 and Uncertain; reviewer correction lowers the score and is logged; tie-breaking: equal score -> higher degree first; a degree never beats a higher score; identical Bachelor's vs Master's resumes (also for uploads).
tests/perfect.test.ts (3)	Ideal LCA consultant written from JD1 scores exactly 100; ideal UI/UX intern with a Bachelor's scores exactly 100; an accountant scores under 15 for both.

A **regression test** is a test written for a bug you already fixed, so it can never quietly come back.

> **SEARCH THIS:** "unit testing explained", "Vitest tutorial", "what is a regression test".

Other checks done: `npm run build` (type check + lint + builds 55 pages, no errors); filters tested in the browser (47 rows -> 10 with problems -> 23 UI/UX; sort by years puts AL-0183 at 5.2 yrs first); upload tested with .docx, .pdf, .txt (friendly error) and an empty file; pages checked at desktop and phone width.

11. Commands cheat sheet

Run these in a terminal opened in the project folder.

Command	What it does
npm install	Download all libraries (first time only, or after pulling changes)
npm run pipeline	Run extract -> review export -> review apply -> score, in order
npm run extract	Step 1 only
npm run review:export / review:apply	Step 2 only
npm run score	Steps 3 and 4 only (fast; use after changing weights)
npm test	Run the 45 tests
npm run dev	Website at http://localhost:3000 with live reload (for editing)
npm run build, then npx next start -p 3123	Production build, then serve it (what we tested at localhost:3123)
npx tsx scripts/set-review.ts AK-4184 review_status=ok "reviewer_notes=..."	Record a review decision from the terminal

> **WATCH OUT:** The website reads results.json **at build time**. After running the pipeline, run npm run build again (or use npm run dev) or the site will show old numbers.

Change a rule, see the effect

1. Edit config/jobs.ts (for example change experience min from 4 to 2 for the LCA role, to follow the JD instead of the brief).
2. Run `npm run score` and read the new top 10 in the terminal.
3. Run `npm test` to make sure nothing broke.
4. Run `npm run dev` and look at the site.

12. Domain words: LCA and UX in plain English

You will be asked about the jobs themselves. Here is enough to talk sensibly.

Sustainability / LCA

Term	Meaning
LCA (Life Cycle Assessment)	Measuring the environmental impact of a product over its whole life: raw materials, making, transport, use, disposal.
LCI (Life Cycle Inventory)	The data part of an LCA: every input (energy, materials) and output (emissions, waste). The JD wants someone who collects and validates this.
LCIA (Impact Assessment)	Turning the inventory into impact numbers like kg CO2-equivalent.
ISO 14040 / 14044	The international standards that say how an LCA must be done.
GaBi, SimaPro, openLCA	LCA software. GaBi (now Sphera) and SimaPro are the paid industry standards named in the JD; openLCA is free.
ecoinvent	The biggest database of background LCA data (e.g. impact of 1 kWh of Indian electricity).
EPD (Environmental Product Declaration)	A published, verified LCA report for a product. 40 EPDs = 40 finished LCA studies.
PCR (Product Category Rules)	Rules for making EPDs for one product type.
Cradle-to-gate / cradle-to-grave	LCA scope: up to leaving the factory / until disposal.
Functional unit, system boundary	What exactly is being compared (e.g. 1 car seat over 10 years) and what is inside the study.
GHG, Scope 1/2/3	Greenhouse gases. Scope 1 = own emissions, 2 = bought electricity, 3 = supply chain and product use.
ESG, BRSR, GRI, CDP	Company sustainability reporting frameworks. Related to LCA but not the same job.
CBAM	EU carbon border tax on imports; needs embedded-emission calculations.

> **SEARCH THIS:** "Life Cycle Assessment explained" (many 5-min videos), "what is an EPD", "scope 1 2 3 emissions explained".

UI/UX design

Term	Meaning
UI vs UX	UI = how screens look (layout, colours, type). UX = how using it feels and whether it works for people.
Figma	The standard design tool for screens and prototypes.
Wireframe	A rough black-and-white sketch of a screen's layout.
Prototype (high-fidelity)	A clickable fake app that looks real, used for testing.
User research, usability testing	Talking to or watching real users to learn what they need and where they struggle.
User flows, information architecture (IA)	The steps a user takes, and how content is organised and labelled.
Design system	Reusable components and rules (buttons, colours, spacing) so a product looks consistent.
Personas, user psychology	Made-up profiles of typical users; understanding why users behave as they do.
Accessibility / WCAG	Designing so people with disabilities can use it; WCAG is the standard checklist.

> **SEARCH THIS:** "UI vs UX explained", "Figma tutorial for beginners", "what is a design system".

13. Decisions and why we made them

Decision	Why	Alternative
Rules, not ML/LLM scoring	47 resumes, no outcome labels; must be explainable and repeatable	Embedding similarity as a small, separate extra (future)
LCA target 4-6 yrs	The assignment brief says 4-6 and is the instruction we are graded on	JD page says 2-3; one setting to change
ISO 14040/44 not scored	Anonymisation removed the numbers from every resume; scoring it would only add noise	Score the word "ISO" near LCA (too loose)
Listed skill = 60%	Claiming is weaker than doing, but not worthless	50% or 70%; small effect on order
Other sustainability years count half	ESG/GHG work is related to LCA but not the same	Ignore them (too harsh) or full (too generous)
50-point line, +/-5 = Needs a look	Simple; borderline cases go to a person instead of a confident wrong label	Per-role thresholds
Intern over-qualification penalty	JD asks for 0-1 yr; a 5-yr designer is unlikely to take an intern role	No penalty
Fixed "today" = Nov 2026	Same input gives the same result on any day	Real today (results drift)
Precomputed JSON, no database	Free Vercel tier, fast, nothing to break; uploads are not saved	Supabase if uploads had to be stored
Keep the empty file in the list	Brief says show errors clearly; hiding it would lose a candidate	Drop it silently
Fix rules, not single rows, when a pattern bug is found	Same bug would hide in other resumes and uploads	Patch rows by hand
UI/UX split 40/40/15/5 (education only 5)	The UI/UX JD stresses tools and output and never mentions a degree	Copying the LCA split (education 15) - the first version, changed after review
Degree as tie-breaker, not points	Identical work: Master's should rank above Bachelor's. But for an internship a degree must not beat better work	Bachelor's 9 / Master's 10 (a weaker Master's could beat a stronger Bachelor's)

14. Interview prep: questions they may ask

Q: Walk me through your solution.

A: Rule-based pipeline: extract facts from each resume with patterns, flag anything unclear, a human review pass with a logged change list, a transparent 4-part score per role with quoted evidence, threshold-based labels, and a Next.js site with an upload route that reuses the same code.

Q: Why didn't you use an LLM to read the resumes?

A: Repeatability and auditability. The same file always gives the same output, every value points to a line in the resume, and it runs free on Vercel. An LLM could help with odd layouts later, but its output would still need the same review step.

Q: How did you verify the extracted data?

A: A risk-sorted review sheet. I checked all flagged and suspicious resumes and the top 5 of each list: 30 of 47. 3 corrected, every change logged old to new in review-log.json, and 8 pattern bugs fixed in code with regression tests.

Q: What is the weakest part?

A: Keyword matching can miss unusual wording, and the weights are judgement calls. Mitigations: evidence quotes, a Problems page, human review, and everything configurable. Also 17 resumes were not hand-checked.

Q: How is the score defined?

A: Section 7: four parts per role adding to 100. Explain the LCA one with TM-4495 as the example.

Q: Why is HM-4257 first and not TM-4495?

A: Both have strong LCA. HM has 4 full years (inside the 4-6 target) and 40 published EPDs; TM has 3.8 years, slightly under target. TM has a better degree match. The difference is 1.3 points; both clearly fit.

Q: The JD says 2-3 years but you used 4-6. Why?

A: The assignment brief, which is what I am evaluated on, says 4-6. I noted the conflict on the site and in the README and made it one setting.

Q: How does upload ranking work?

A: Same extract, score and classify functions; rank = 1 + number of pile resumes with a higher score. Nothing is saved.

Q: What happens with a scanned PDF?

A: No text layer means no text, so it is flagged unreadable and scores 0 with Needs a look. OCR (e.g. Tesseract) would be the next step.

Q: How would you scale to 10,000 resumes?

A: Move results to a database (e.g. Supabase/Postgres), run extraction in a background job queue, add pagination, keep the scoring functions as they are.

Q: How did you choose the weights?

A: The JDs give no weights, so I followed what each JD stresses. LCA: skills 40, years 25, the 3-project rule 20, education 15 (the JD has a degree line). UI/UX: skills 40, design work 40 (the job is about output), experience fit 15, education 5 (the JD never mentions a degree). Must-haves are 60% of skills; doing beats listing (listed = 60%). An ideal-candidate test must score 100.

Q: How would you improve accuracy?

A: Larger synonym list from more resumes, an optional embedding similarity score shown separately, a small labelled set from recruiters to tune weights, and a UI for reviewers to correct data directly.

Q: What did AI do and what did you do?

A: Answer honestly: AI (Claude) helped write code and suggested review decisions. I made each review decision after reading the original, chose the trade-offs, and checked the results. Know this guide well enough to explain any part yourself.

Q: Is it fair to candidates?

A: It never uses names (anonymised), scores only job-related evidence, shows its reasons, and sends borderline or broken cases to a person instead of rejecting them.

15. Known limits and what to improve next

- Keyword patterns can miss unusual wording or catch the wrong meaning (8 such bugs already found and fixed).
- Weights and thresholds are reasoned guesses, not tuned on hiring outcomes.
- 17 of 47 resumes not hand-checked (low risk: no warnings, not in the top 5).
- ISO standards and portfolio links cannot be checked because anonymisation removed them.
- Scanned PDFs (images) cannot be read without OCR.
- Uploaded resumes are not reviewed or saved; a real product would store them and let a reviewer correct them.
- No login: anyone with the link can see the rankings (fine for a prototype with anonymised data).

Ideas if they ask "what with more time?"

- Reviewer edits in the website instead of a CSV (save to Supabase).
- Optional semantic match score using embeddings, shown as its own line, max 10%.
- Per-recruiter weight sliders on the How-it-works page.
- OCR for scanned PDFs; LinkedIn/portfolio link checks.
- Bias checks: compare score distributions across groups once real data exists.

16. Glossary with search tags

Word	Meaning	Search this
API / route handler	An address a program calls to get data back	what is a REST API
Build	Turning source code into the fast files a server sends	what does npm run build do
CSV	Spreadsheet as plain text, values separated by commas	CSV file explained
Component	A reusable piece of a web page	React components explained
Deploy	Putting the site on a public server	deploy Next.js Vercel
Evidence	The quote from the resume that justifies a point	-
Extraction	Pulling structured facts out of messy text	information extraction NLP
False positive	The rule matched something it should not have (car "usability")	false positive vs false negative
FormData	How a browser sends a file to a server	FormData file upload fetch
Heuristic	A rule of thumb that is usually right	heuristic meaning programming
Human-in-the-loop	A person checks or corrects the machine's output	human in the loop AI
JSON	Text format for data: {"score": 87.1}	JSON in 100 seconds
Lint	Automatic style and mistake checker	what is ESLint
Pipeline	Steps run in order, each feeding the next	data pipeline explained
Regex	Pattern for finding text	Regular expressions in 100 seconds; regex101.com
Regression test	Test that keeps a fixed bug fixed	regression testing explained
Serverless	Code that runs on demand on the host's machines	serverless explained Fireship
Static site / SSG	Pages built ahead of time	Next.js static site generation
Synonym dictionary	List of spellings for one skill	-
Threshold	The cut-off line (50 points)	-
TypeScript type	A label saying what shape data has	TypeScript types tutorial
Unit test	Small automatic check of one function	unit testing for beginners

17. What is done, what is left, how to review

Done

- Steps 1-5 built and tested (45 tests incl. ideal-candidate check, build passes, browser-tested).
- Human review of 30/47 with logged decisions; 3 corrections; 8 rule bugs fixed.
- README.md (methodology) and docs/methodology.md (form text).

Left (in order)

1. **You**: read this guide, the README and docs/methodology.md. Change anything that does not sound like you.
2. **You**: click through the site (5-point test below). Tell Claude anything confusing.
3. **Optional**: review the remaining 17 resumes (about 30 min).
4. **Claude, with your OK**: git commit + push to a GitHub repo under SoAkeD1.
5. **When you say so**: deploy to Vercel (you log in yourself). Put the link in the README.
6. **You**: fill the GFI form (email, phone, github.com/SoAkeD1, start date, Vercel link, method text) and do the quiz. Deadline 14 Nov 2026.

5-point website test

1. UI/UX tab: is DVJ-0268 #1?
2. Open HM-4257, expand "Show proof" under LCA projects: do you see "40 EPDs"?
3. Tick "Only show resumes with problems": 10 rows?
4. Upload any file from public/resumes: does it say it is already in the pile?
5. Problems page: is resume-0327 at the top?

Suggested review order for the code

1. config/jobs.ts (what we look for) -> lib/skills.ts (the dictionary).
2. lib/score.ts (the scoring rules) with section 7 open beside it.
3. lib/classify.ts (short).
4. lib/dates.ts with tests/dates.test.ts.
5. lib/extract.ts (longest; read the comments, numbered 1-10).
6. data/review-log.json (your review), then the website pages.

> **CAVEMAN SAYS:** If you understand sections 2, 5, 6, 7 and 13, you understand the project. The rest is detail.